

2025–2026

3D Graphics Project

HDR Bloom Rendering for a Real-Time Solar System Simulation

Niccolò Benedetto 626872
Paolo Alberto Bordis 628838

Prof. Daniele Bonatto
PhD Eline Soetens

June 2026

Contents

1 Introduction **1**

2 Scene Architecture and Rendering Pipeline **2**

2.1 Phong Lighting Model 3

2.2 HDR Rendering and Bloom Effect 3

2.3 Earth Night Lights 4

2.4 Reflection Mapping 4

3 Environment and Additional Features **5**

3.1 Skybox Environment 5

3.2 Asteroid Field and OBJ Loading 5

3.3 Interactive Controls and Camera System 5

4 Conclusion **6**

1 Introduction

This project develops a real 3D Solar System renderer using OpenGL and C++. The scene simulates a simplified but visually coherent representation of the Solar System, including the Sun, the eights planets, the Earth's moon, orbital trajectories and a field of asteroids.

The main feature of the project is the bloom rendering pipeline. Visually speaking the Sun acts as main emissive light source and produces a glow wffect through HDR rendering on the other objects. We implemented as well additional features as Earth night lights, cubemap skybox and reflection.

The scene supports free camera navigation, orbital animations, textured models, and runtime manipulation of bloom and exposure parameters through a graphical user interface (ImGui). The complete list of features is listed in the `readme.md` file in the GitHub repository.

Speaking of the techical components the project was implemented using OpenGL 3.3 Core Profile in C++. In addition we take into account several external libraries, including GLFW for window and input manipulation, GLAD for OpenGL function loading, GLM for mathematical operations, Dear ImGui for runtime graphical controls, and TinyObjLoader for OBJ model loading.

The complete source code of the project is available on GitHub, includng the instruction to download and run the program:

https://github.com/NiccoBene00/OpenGL-SolarSystemSceneRendering

The project was developed and tested on the following hardware and software configuration:

Component	**Specification**
GPU Vendor	Intel
GPU Renderer	Intel(R) Iris(R) Xe Graphics
OpenGL Version	3.3.0
Driver Build	31.0.101.4032

Table 1.1: Graphics environment used for rendering and testing.

2 Scene Architecture and Rendering Pipeline

Each planet is rendered using textured sphere meshes together with transformation matrices that control translation, rotation, and scaling.

In addition each planet rotates around the Sun using time-dependent orbital equations, while the Moon rotates around the Earth using a secondary orbital transformation. We implemented then orbiatl trajectories by means circular line meshes rendered independently from the planets geometry.

About the bloom rendering pipeline we exploit a multiple frame buffer architecture: during the first rendering step, the complete 3D scene is rendered into a floating-point framebuffer object (FBO) containing two color attachments. The first attachment stores the complete HDR scene, while the second attachment extracts only the brightest pixels required for the bloom effect. This separation between scene rendering and luminance extraction allows the bloom effect to affect only high-energy regions of the image while preserving the details of darker areas.

At this point bright regions are processed using a ping-pong Gaussian blur technique implemented through two additional framebuffers: one for the horizontal blur step and the other one for the vertical in order to produce a nice glow effect around the emissive lights source such as the sun.

At the end of the day a post-processing step combines the original HDR scene with the blurred bloom texture. In order to get the final outcome we applied as well a tone mapping step, to display correctly color through the screen. The project also implements a free-roaming camera system based on GLFW mouse and keyboard input.

The main pipeline for the achievement of the bloom effect can be summarize through the netx figure shows:

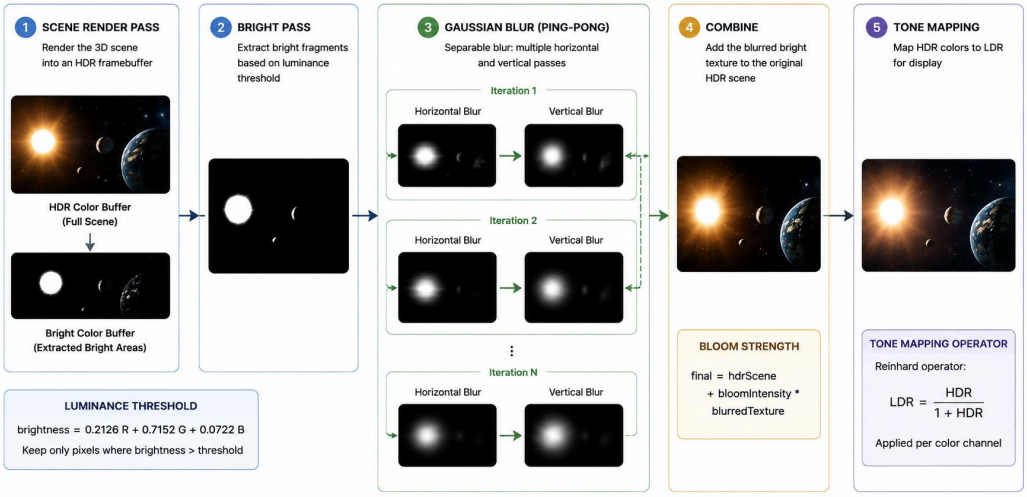


Figure 2.1: Bloom Effect Pipeline

2.1 Phong Lighting Model

All the objects that receives light (non-emissive objects) in the scene are rendered using the Phong lighting model. The illumination of each fragment is computed as the sum of ambient, diffuse, and specular components:

$$I = I_{ambient} + I_{diffuse} + I_{specular}$$

Diffuse lighting is calculated using the Lambert cosine law:

$$I_{diffuse} = k_d \cdot \max(\mathbf{N} \cdot \mathbf{L}, 0)$$

where $\mathbf{N}$ is the surface normal of the non-emissive object and $\mathbf{L}$ is the light direction vector point toward the light source (emissive object). Specular reflections (bright highlight from reflective/shiny surfaces) are computed using the view direction and reflection vector to simulate shiny surfaces.

2.2 HDR Rendering and Bloom Effect

The implementation is based on a floating-point HDR framebuffer with two color attachments. During the rendering pass, the scene is rendered normally while simultaneously extracting only the brightest fragments using a luminance threshold:

$$brightness = 0.2126R + 0.7152G + 0.0722B$$

Only fragments above a specified threshold are stored inside the bright color buffer.

The extracted bright texture is then processed using a Gaussian blur filter (for each pixel we consider the neighboor and we computed a weighted sum) implemented with a ping-pong framebuffer technique. Multiple horizontal and vertical blur passes are alternated to create a smooth glow effect.

2.3 Earth Night Lights

In order to increase realism, the Earth uses an additional emissive night texture representing illuminated cities visible on the dark side of the planet. We achieved suche effect by uusing a secondary texture map combined with the diffuse lighting term.

The visibility of the night texture depends on the amount of direct illumination received from the Sun. Regions facing away from the light source gradually produce a smooth transition to the city lights.

2.4 Reflection Mapping

The project also integrates basic environment reflections using cubemap reflection mapping. Reflection vectors are computed inside the fragment shader using the OpenGL `reflect()` function and are used to sample the skybox cubemap texture on the Earth planet.

3 Environment and Additional Features

In addition to the main rendering pipeline, we implemented other features in order to make the scene as realistic as possible.

3.1 Skybox Environment

The surrounding space environment is rendered using a cubemap skybox composed of six high-resolution textures representing deep space. The skybox is rendered as a cube centered around the camera position with the constraint of removing the translational component of the view matrix. This technique creates the illusion of an infinitely distant environment.

3.2 Asteroid Field and OBJ Loading

In order to enhance the external environment of the Solar System we thought to generate an asteroids field. Hence we rendere multiple asteroid instances by using random positions, scales, and rotation parameters.

The asteroid geometry is loaded as OBJ models together with associated MTL material files and diffuse textures. OBJ loading was implemented using the `Tiny Object Loader` library, while texture loading is handled through stb_image.

3.3 Interactive Controls and Camera System

The project includes as well a free-roaming camera system controlled through mouse and keyboard input. So doing users are allowed to freely navigate the scene and inspect planets, lighting effects, and environmental details from arbitrary viewpoints.

A runtime graphical user interface was implemented using `Dear ImGui`. The interface allows real-time modification of simulation parameters such as orbital speed, bloom activation, exposure intensity, and orbit visibility.

4 Conclusion

The visual outcome ensures as the combination of floating point framebuffers, bight-step extraction, Gaussian blur filterand tone mapping (for display correctly HDR colors) allows the implementation of a real bloom-glow effect by means light diffusion from the ligh source. The remain additional rendering architecture exploits instead Phong illuminatio model, cubemap environment, texture OBJ loading and orbital and sphere transformations. Therefore the project demonstrates how modern OpenGL rendering pipelines can combine real-time lighting, HDR rendering, and post-processing techniques to produce visually coherent and interactive planetary simulations.